HTTP Parameter Pollution

Julian Liborius Simon Wöhltjen Danje Petersen

TU Braunschweig, Germany

June 6, 2024

HTTP Parameter Pollution

- Exploit wurde 2009 öffentlich gemacht
- Client-side
- **TODO** ▶*basic facts über den exploit*◀

HTTP Parameter

- Senden Informationen vom Klienten zum Server
- Können an verschiedenen Orten versendet werden
  - im Query-Teil der URI (GET request)
    → `www.example.org?name=Manfred&age=44`
  - im request body (POST request)
  - in HTTP headern

Parameter Precedence

- Problem entsteht, wenn für einen Parameter genau ein Wert erwartet wird, aber mehrere erhalten werden
- Je nach Programmiersprache und Server wird unterschiedlich vorgegangen

Technology/Server	Tested Method	Parameter Precedence
ASP/IIS	`Request.QueryString("par")`	All (comma-delimited string)
PHP/Apache	`$_GET["par"]`	Last
JSP/Tomcat	`Request.getParameter("par")`	First
Perl(CGI)/Apache	`Param("par")`	First
Python/Apache	`getvalue("par")`	All (List)

- So können falsche Parameter in die URI hinzugefügt werden, die die Richtigen überschreiben

Parameter Pollution

- Neben dem zu manipulierenden Parameter kann es auch einen Parameter geben der polluted wird.
- Dafür wird in diesen der manipulierte Parameter in verschlüsselter Form eingefügt
- Reservierte Zeichen in den value-strings müssen in Hexadezimalform kodiert sein
- z.B.: %26 für „&“, %3D für „=“
- So lässt sich ein *name=value* Paar als *%26name%3Dvalue* einfügen
- Die Parameter werden bei Benutzung des manipulierten Links dekodiert und es entstehen Duplikate

Parameter Precedence Beispiel

- Angenommen, es wird immer der erste Parameterwert genutzt

  `example.org?name=Manfred&age=44`

- Der erste Parameter wird „polluted“

  `example.org?name=Manfred%26age%3D20`

- Wenn nun das richtige Alter eingegeben wird, werden folgende Parameter verschickt:

  `example.org?name=Manfred&age=20&age=44`